

LLM-Guided Evolution Setup Guide for PACE ICE

This guide walks you through setting up and running the llm-guided-evolution project on Georgia Tech's PACE ICE cluster.

Table of Contents

- Accessing PACE ICE
- Cloning the Repository
- Setting Up the Virtual Environment
- Preparing Slurm Scripts
- Testing with Datasets
- Running the Project
- Job Chaining for Long Runs

Accessing PACE ICE

Connect to the cluster via SSH:

```
ssh username@login-ice.pace.gatech.edu
```

Enter your Georgia Tech password when prompted.

Important: Do not run intensive tasks on login nodes. Request a compute node first:

```
srun -G 1 --pty bash
```

Cloning the Repository

MAKE SURE TO CLONE INTO SCRATCH!

pwd should output /home/hice1/<username>/scratch/ before you use the clone command

Clone the MosesClean branch from the repository.

Repository: <https://github.com/loriencho/llm-guided-evolution-infrastructure/tree/MosesClean>

Option 1: HTTPS

```
git clone -b MosesClean https://github.com/loriencho/llm-guided-evolution-infrastructure.git
```

You'll need your GitHub account credentials.

Option 2: SSH

```
git clone -b MosesClean git@github.com:loriencho/llm-guided-evolution-infrastructure.git
```

Note: If using SSH, you may need to generate an SSH key on PACE ICE and add it to your GitHub account:

Setting Up the Virtual Environment

Option 1: Using uv (Recommended)

1. Install uv:
`curl -LsSf https://astral.sh/uv/install.sh | sh`
Or follow: <https://docs.astral.sh/uv/getting-started/installation/>
2. Navigate to the project root (the directory containing pyproject.toml):
`cd llm-guided-evolution-infrastructure`
3. Create and sync the virtual environment:
`uv sync --cache-dir ~/scratch/.uv`
Note: The `--cache-dir` flag is important on PACE ICE due to limited home directory quota. The scratch partition has more space.

Option 2: Using Conda

If using conda, you'll need to install dependencies from a requirements.txt file.

- Use the requirements from:
<https://github.com/jasonzutty/llm-guided-evolution-fork/commit/f773e6cdc36e40e590df7e767f39500d2e393902>
- Alternatively, generate a requirements.txt from pyproject.toml

Preparing Slurm Scripts

Before running the project, you must generate the Slurm batch scripts:

1. Ensure your configuration is set (see Configuration section below)
2. Generate the scripts:
`uv run slurm.py`

This generates:

- `run.sh` - Single evolution run script
- `server.sh` - LLM inference server script
- `island_controller.sbatch` - Island migration controller
- `src/mixt.sh` - Crossover script

- slurm-config/slurm_config.yaml - Runtime configuration
3. Create output directories (if they don't exist):
`mkdir -p run_job_outputs/{server,islands,evaluation,evolution}`

Important: You must re-run `uv run slurm.py` whenever you change configurations or switch between datasets.

Testing with Datasets

Titanic Dataset

1. Set up Kaggle API credentials:
Follow instructions at:
<https://github.com/Kaggle/kaggle-api/blob/main/docs/README.md#api-credentials>
2. Download the data:
`cd sota/Titanica/Titanic`
`uv run ./pull_data.sh`
Or manually place train.csv in sota/Titanic/data/
3. Preprocess the data:
`uv run preprocess.py`
This creates data/processed_train.csv
4. Test the setup:
`uv run eval.py`

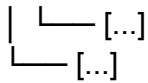
For full details, see: `sota/Titanic/readme.md`

CIFAR-10 Dataset

Reference documentation:

<https://github.com/jasonzutt/llm-guided-evolution-fork/blob/vip-docs-second-rebase/sota/ExquisiteNetV2/README.md>

1. Download CIFAR-10:
Download cifar-10-python into your project root directory.
2. Run the split script:
`c .`
`uv run split.py`
3. Verify your directory structure:
llm-guided-evolution-Island-Migration/
├── cifar10/ | └── [...]
├── sota/ExquisiteNetV2
| └── [...]
└── src/



Running the Project

Configuration

The constants.py file is a symbolic link that controls which configuration is used.

Switch to titanic configuration:

In -sf constants_titanic.py src/cfg/constants.py

Switch to island migration configuration:

In -sf constants_island_migration.py src/cfg/constants.py

Important: After changing the symlink, regenerate Slurm scripts:

uv run slurm.py

Running with Islands (Default)

1. Submit the server job:
sbatch server.sh N
This starts the LLM inference server and automatically submits the island controller.
N is # of “eras” (ctrl-f era in islands_wrapper.py)
2. Monitor the output:
tail -f run_job_outputs/server/slurm-<job_id>.out
or
Find the file and open it

Running without Islands

To run a single evolution without islands:

1. Modify line 36 of server.sh to use run.sh instead of island_controller.sbatch

Troubleshooting

- Module not found errors: Ensure you're using uv run prefix for all Python commands
- SSH cloning fails: Verify your SSH key is properly added to GitHub
- Job fails immediately: Check that output directories exist (run_job_outputs/)
- Disk quota exceeded: Ensure you've cloned to your scratch directory not your home.
- Job out of quota: Use --cache-dir ~/scratch/.uv with uv commands

- Wrong configuration: Verify `src/cfg/constants.py` symlink points to correct file and re-run `uv run slurm.py`

Additional Exercises

If you have completed all the tasks above, here are some small challenges to ensure you've got a stronger understanding of the codebase

1. Change the SOTA model for titanic to a ML or GP model you used during bootcamp
2. Write your own prompts for LLMGE to use to evolve models
3. Use `pareto_front_images.py` and `pareto_front_gif.py` to create an animation of an existing run's evolution. (may need editing)